(A.1) Projectplan - Lokaal Back-up Systeem

Gemaakt door Senna Draaijer Datum: 21 / 09 / 2026

Inleiding

Voor het nieuwe schoolproject hebben wij opdracht gekregen om zelf te kiezen om een project te maken, dit project zal moeten goedgekeurd worden door de opdrachtgever (docent). Ik heb zelf gekozen om een lokaal back-up systeem (Kort voor: "LBS") te maken, om verschillende bestanden op te slaan/ophalen.

Projectomschrijving

Een back-up systeem is een systeem waar je bestanden kan op slaan, dit soort programma's zijn er voor om dataverlies te voorkomen. Dit systeem zal een command line interface programma zijn, waardoor er geen geen website of een visuele applicatie zal zijn maar wel een interface om te zien wat je daadwerkelijk op zal slaan.

In het programma kan je verschillende bestand(en) of map(pen) op slaan, de bestanden die je wilt opslaan zullen in een *backupbestand* komen waar het pad van het bestand(en)/folder(s) in dit bestand zal komen staan. Hierdoor kan je dus ook meerdere back-ups maken.

Ook zal je deze back-up's terug kunnen halen op basis van de naam van het *backupbestand* hierdoor zullen de bestanden overscreven worden. Er komt hiervoor natuurlijk wel een waarschuwing.

De *backupbestanden* zal je ook handmatig kunnen bewerken, om ze te openen kan je de bestandspaden vinden van alle aangemaakte *backupbestanden*. Deze bestanden zullen ook een bestand er naast hebben om de versiebeheer bij te houden. Dus je kan ook oudere versies kunnen ophalen op basis van zo'n versiebeheer bestand.

Om het systeem te gebruiken zal er ook een handleiding zijn om de gebruiker te helpen, dit kan worden aangeroepen als argument in het programma (denk aan "git --help"). Ook zal er een website beschiktbaar zijn om de documentatie van het programma te lezen, waar er meer wordt uigelegd over het programma.

Doelstellingen

De gebruiker kan automatisch opslaan meerdere bestanden op slaan, in plaats van handmatig de bestanden een voor een te kopieren. Als de gebruiker hulp nodig heeft kan hij het hulp commando oproepen in plaats van de source code te lezen. Om meer te leren over back-up systemen & versiebeheer.

Resultaat

De gebruiker zal een *backupbestand* kunnen aanmaken om daar vervolgens bestanden en folders aan te koppelen. Zodra de gebruiker bestanden heeft toegevoegd in het *backupbestand* zal de gebruiker een back-up kunnen maken van de bestanden/folders die in het *backupbestand* staan. Dan maakt het programma een kopie van alle bestanden en zet ze in een aparte folder met dezelfde naam als het *backupbestand*.

Zodra er een back-up is gemaakt zal de gebruiker de back-up ook kunnen ophalen, zodra de gebruiker dit doet zal hij een bevestiging krijgen dat hij moet accepteren. Als dit geaccepteerd wordt zullen de bestanden worden vervangen waar de oude versies stonden.

Daarnaast zal je verschillende versies kunnen ophalen van back-ups, dus een versiebeheer alleen zonder veranderings tekst (wat github doet met commits). De gebruiker kan ook oudere back-ups terughalen om bijvoorbeeld een werkende versie op te halen. Deze kan je ophalen met een versie. Elke back-up voegd een 0.1 bij de versie. waar dan versie "0.9 + 0.1 = versie: 1.0".

Samenvatting Resultaten

- *backupbestand* aanmaken
- folder(s) en bestand(en) toevoegen aan *backupbestand*
- backup uitvoeren gebaseerd op geselecteerde *backupbestand*
- backup ophalen gebaseerd op geselecteerde *backupbestand*
- versiebeheer ophalen van geselecteerde *backupbestand*
- gemakkelijk backup versies terughalen van bestanden met behulp van een versie-beheer bestand.
- *backupbestanden* handmatig kunnen aanpassen
- hulp pagina kunnen aanroepen

Afbakening

MoSCoW

ID	EIS	Prioriteit
E - 1	Programma aan roepen vanuit terminal \	Bash

ID	EIS	Prioriteit
E - 2	*Backupbestand* aanmaken	Must-Have
E - 3	Bestand toevoegen aan *backupbestand*	Must-Have
E - 4	Back-up aanmaken gebaseerd op *backupbestand*	Must-Have
E - 5	Bestand verwijderen uit *backupbestand*	Must-Have
E - 6	Hulp-pagina oproepen	Must-Have
E - 7	*backupbestanden* ophalen	Should-Have
E - 8	Folder toevoegen aan *backupbestand*	Should-Have
E - 9	Folder verwijderen uit *backupbestand*	Should-Have
E - 10	Bevestiging zodra een back-up wordt opgehaald	Should-Have
E - 11	Versiebeheer	Should-Have
E - 12	Commit-bericht voor backups	Could-Have
E - 13	Documentatie website	Could-Have
E - 14	Meerdere Folders/Bestanden toevoegen aan *backupbestand*	Could-Have
E - 15	Automatische Backups	Could-Have
E - 16	Meerdere Folders/Bestanden verwijderen uit *backupbestand*	Wont-Have
E - 17	Cloud Opslag	Wont-Have
E - 18	User interface (Volledige visuele applicatie)	Wont-Have

Randvoorwaarde

- Weekelijkse meeting met productowner *Om de progressie van het project te laten zien Om feedback te ontvangen en gebruiken Om het project te testen*
- Github Repository *Om versiebeheer van dit project bij te houden*
- Docenten *Om vragen tegen te stellen voor advies en feedback*

Risicoanalyse

- Github servers zijn inactief *Geen actieve versiebeheer en mogelijke staking van het werk door niet werkend product.*
- Ziekte (bij student) *Product zal geen voortgang zien.*
- Ziekte (bij productowner) *Geen feedback kunnen krijgen van productowner en meetings kunnen geannuleerd worden.*
- Backup verwijderd systeem bestanden *kan leiden tot tijdverlies dat tussen een uur tot een gehele dag kosten.*

Fasering

Fase	Periode	Belangrijkste Activiteiten	OpleverProduct
1. Requirements & Analyse	Week 1	Vaststellen van benodigde eisen (functioneel en niet-functioneel).	Programma van Eisen (PvE)
2. Design	Week 1	Het product gaan ontwerpen, user-flow ontwerpen.	Functioneel & Technisch Ontwerp
3. Realisatie	Week 2 - 4	De benodigde functies bouwen en dit samenstellen tot een systeem.	Werkend prototype

Fase	Periode	Belangrijkste Activiteiten	OpleverProduct
4. Testing	Week 2 - 4	Het product laten testen door willekeurige mensen en de product owner op basis van de test-cases	Test-rapport(en)
5. Oplevering	Week 4	Overdragen van het project naar de product-owner.	Oplevermoment

Planning

Voor dit project is vastgesteld 5 weken, dit is inclusief de herfstvakantie. De planning gaat er van uit dat er **NIET** doorgewerkt wordt in de herfstvakantie dus 4 weken.

Dag	Taak	Uren
21-9-26	Projectplan	4
25-9-26	Functioneel ontwerp	4
26-9-26	Technisch ontwerp	2
27-9-26	Testcases	2
28-9-26	testplannen maken	3
28-9-26	Programmeren	8
30-9-26	Programmeren	8
2-10-26	Programmeren	8
5-10-26	programmeren	4
7-10-26	Testen	2
7-10-26	Testrapporten typen	4
9-10-26	programmeren	8
	Herfstvakantie	-
19-10-26	Bufferdag	-
21-10-26	Bufferdag	-
23-10-26	Bufferdag	-
25-10-26	Opleveren van Product	4